

Part A: Relation Properties

In the following section, we will assess the properties of relations depicted as matrices. The function will receive only one input, the adjacency matrix of the graph representing the relation, for example:

- ```
adj_mat = [[1,0,1],
 [0,0,1],
 [1,1,1]]
```

Here the relation can be represented by the pairs  $\{(1,1), (1,3), (2,3), (3,1), (3,2), (3,3)\}$  since an element of the adjacency matrix in the  $i^{\text{th}}$  row and  $j^{\text{th}}$  column is only 1 if the pair  $(i,j)$  is in the relation

- (a) Write a function to check whether an inputted relation is reflexive. The python function should be called `is_reflexive`, should take in the adjacency matrix argument representing the relation, and should return a boolean, `True` or `False`.

```
def is_reflexive(adj_mat):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (b) Write a function to check whether an inputted relation is symmetric. The python function should be called `is_symmetric`, should take in the adjacency matrix arguments representing the relation, and should return a boolean, `True` or `False`.

```
def is_symmetric(adj_mat):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (c) Write a function to check whether an inputted relation is transitive. The python function should be called `is_transitive`, should take in the adjacency matrix arguments representing the relation, and should return a boolean, `True` or `False`.

```
def is_transitive(adj_mat):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (d) Write a function to check whether an inputted relation is irreflexive. The python function should be called `is_irreflexive`, should take in the adjacency matrix arguments representing the relation, and should return a boolean, `True` or `False`.

```
def is_irreflexive(adj_mat):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (e) Write a function to check whether an inputted relation is antisymmetric. The python function should be called `is_antisymmetric`, should take in the adjacency matrix arguments representing the relation, and should return a boolean, `True` or `False`.

```
def is_antisymmetric(adj_mat):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

Note: We did not provide any test cases for antisymmetric or irreflexive, as we would like you to try and test it yourself. Writing test cases is an important part of coding and it is good practice not learn how to test your code. Please feel free to test your code with whatever examples you could come up with, but when you submit DO NOT ALTER THE MAIN METHOD

## Part B: Function Properties

In the following sections we will assess the properties of an inputted function. The function will be inputted in 3 parts:

- A list of the values in the domain For example:

```
domain = [1,2,3,4]
```

- A list of the values in the co-domain

```
co_domain = [1,2,3,4,5,6,7]
```

- A dictionary of the function mapping between the domain and the co-domain For example:

```
mapping = {
 1: 2,
 2: 3,
 3: 1,
 4: 3
}
```

The example domain, co-domain, and mapping given above would represent this function:

- (a) Write a function to check whether an inputted function is onto. The python function should be called `is_onto`, should take in the 3 arguments representing the function (domain, co-domain, and mapping), and should return a boolean, `True` or `False`.

```
def is_onto(domain,co_domain,mapping):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (b) Write a function to check whether an inputted function is one-to-one. The python function should be called `is_one_to_one`, should take in the 3 arguments representing the function (domain, co-domain, and mapping), and should return a boolean, `True` or `False`.

```
def is_one_to_one(domain,co_domain,mapping):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (c) Write a function to check whether an inputted function is bijective. (Hint: you can utilize your other functions in writing this one!) The python function should be called `is_bijective`, should take in the 3 arguments representing the function (domain, co-domain, and mapping) and should return a boolean, `True` or `False`.

```
def is_bijective(domain,co_domain,mapping):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```